METAHEURISTIK

Algoritma JAYA dan Simulated Annealing (SA)

Kelompok 4 Kelas C

I Dewa Made Satya Raditya
5027231051

Ahmad Wildan Fawwaz
5027241001

Muhammad Rakha Hananditya Rauf
5027241015

Theodorus Aaron Ugraha
5027241056

M. Hikari Reiziq Rakhmadinta
5027241079



DAFTAR ISI

Algoritma JAYA

- Definisi
- Karakteristik
- Keunggulan & Kelemahan
- Cara Kerja

Algoritma SA

- Definisi
- Karakteristik
- Keunggulan & Kelemahan
- Cara Kerja

Definisi JAYA

JAYA adalah algoritma metaheuristika berbasis populasi yang diperkenalkan oleh R.V. Rao pada tahun 2016.

Algoritma ini berasal dari bahasa Sansekerta yang berarti Kemenangan atau Keberhasilan untuk mencapai solusi terbaik.

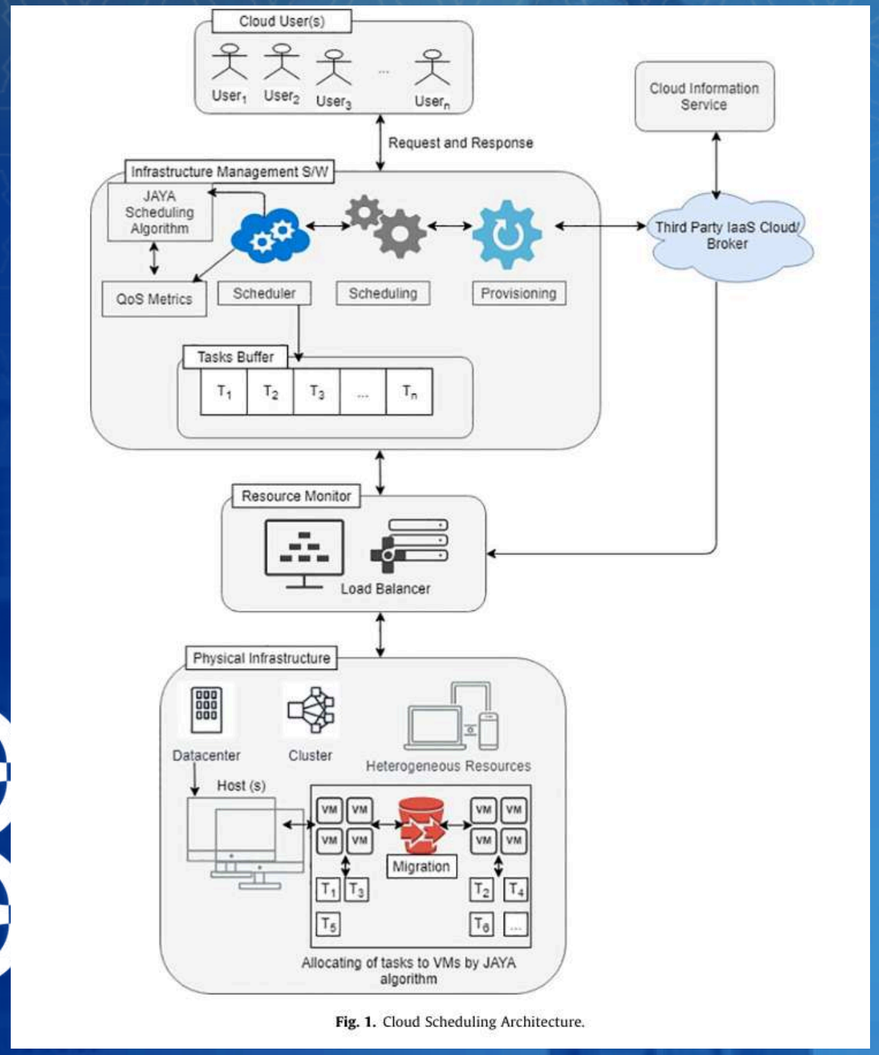


Fig. 1. Cloud Scheduling Architecture.

Filosofi JAYA

Mendekati Solusi Terbaik: Bergerak menuju kandidat terbaik.

Menjauhi Solusi Terburuk: Menghindari lokasi kandidat terburuk.

Karakteristik JAYA

Bebas Parameter Spesifik

Tanpa tuning parameter spesifik algoritma.

Berbasis Populasi

Mengelola banyak kandidat solusi bersamaan.

Konvergensi Cepat

Efisiensi komputasi tinggi untuk fungsi kompleks.

Mudah Diimplementasi

Logika pembaruan sederhana dan efisien.



Definisi SA

Simulated Annealing atau **SA** adalah algoritma metaheuristika berbasis solusi tunggal yang disimulasikan dari proses fisik pendinginan logam.

Algoritma ini berasal dari inspirasi yang diambil dari salah satu teori fisika (Metalurgi), yaitu Proses memanaskan logam hingga suhu tinggi lalu medinginkannya perlahan untuk mencapai struktur kristal berenergi minimum.

Karakteristik SA

Single-Solution

Memproses satu titik solusi dalam satu waktu.

Kriteria Metropolis

Menerima solusi buruk berdasarkan probabilitas $P = e^{-\Delta E/T}$.

Jadwal Pendinginan

Suhu diturunkan bertahap (misal $T = \alpha \times T$).

Penggunaan Memori Rendah

Sangat hemat memori karena tanpa populasi.



Keunggulan JAYA

- **Unggul di Makespan dan Resource Utilization**

task selesai lebih cepat dan VM terpakai lebih merata dibanding GA dan BPSO.

- **Bebas Parameter Spesifik**

tidak perlu tuning khusus seperti SA, cukup ukuran populasi dan jumlah iterasi.

- **Simpel**

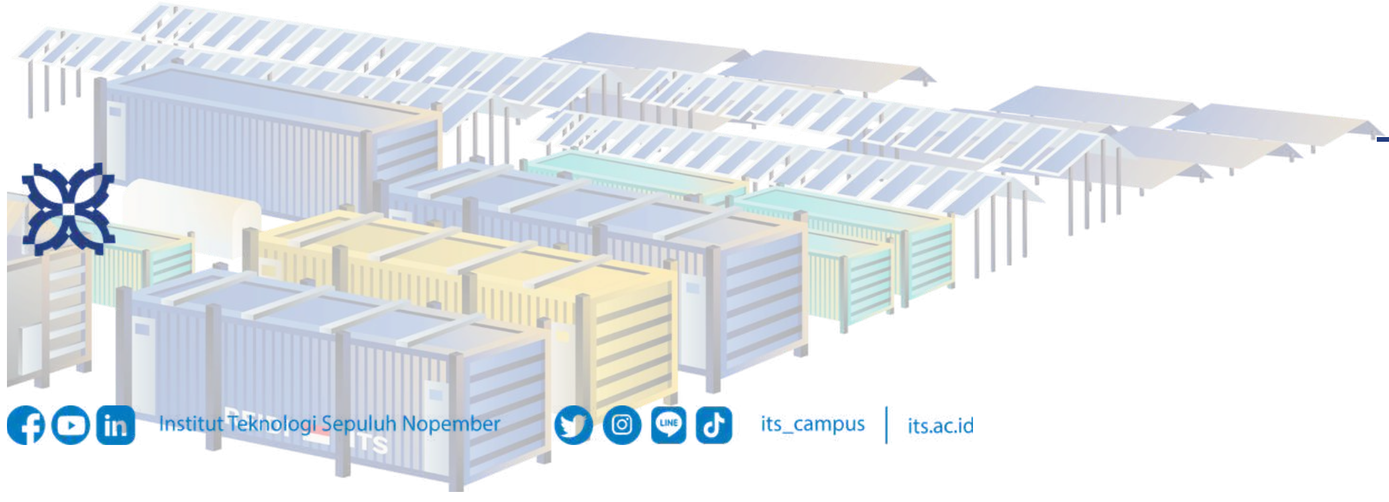
satu rumus pembaruan, tanpa crossover atau mutasi.

- **Eksplorasi dan Eksploitasi Seimbang**

mendekati solusi terbaik sambil menjauhi solusi terburuk.

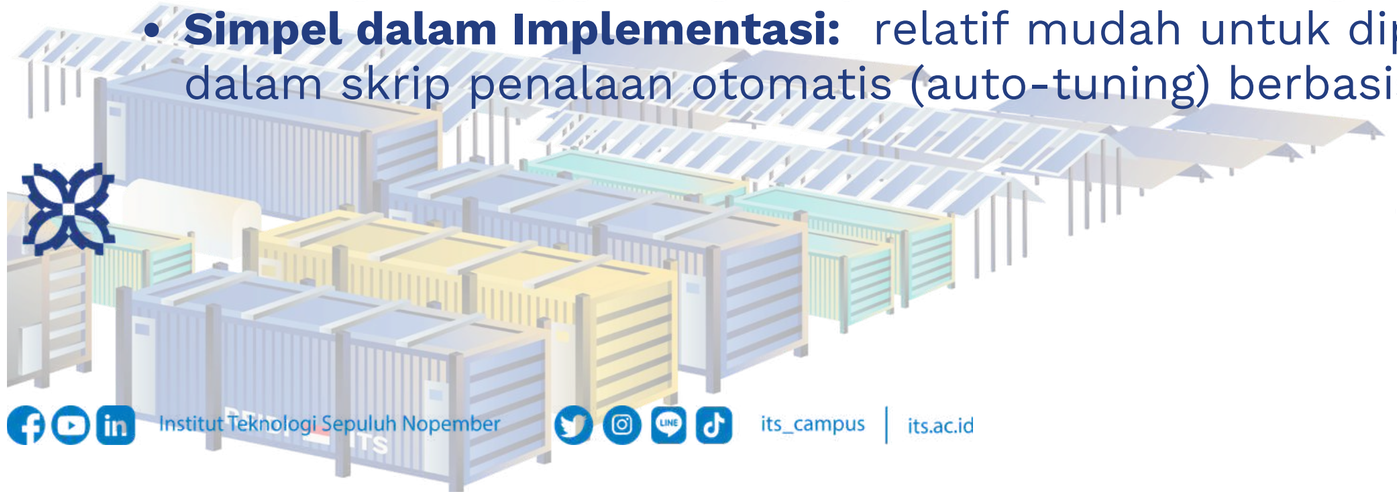
- **Fleksibel**

fitness bisa memuat makespan, utilisasi, dan energi, serta mudah digabung load balancing.



Keunggulan SA

- **Menghindari Optimum Lokal:**
  - algoritme ini secara sengaja menerima hasil yang lebih buruk pada tahap awal, ia mampu melewati penurunan performa demi menemukan tingkat efisiensi setinggi mungkin.
- **Menangani Kompleksitas Non-Linear:**
  - tidak memerlukan ruang yang secara matematis mulus atau dapat diturunkan (diferensiabel) agar dapat berfungsi secara efektif.
- **Fleksibel:**
  - mampu mengoptimalkan berbagai tujuan yang saling bertentangan secara bersamaan seperti memaksimalkan throughput sekaligus meminimalkan biaya komputasi awan dengan menggabungkannya ke dalam satu fungsi biaya.
- **Simpel dalam Implementasi:** relatif mudah untuk diprogram, dikonfigurasi, dan diterapkan dalam skrip penalaan otomatis (auto-tuning) berbasis cloud yang mendasar.



Kelemahan JAYA

- **Mudah Terjebak Optimum Lokal**

semua kandidat mengejar solusi terbaik yang sama, jadi variasi cepat habis.

- **Perlu Konversi Kontinu ke Diskret**

hasil JAYA berupa desimal, padahal nomor VM harus bilangan bulat.

- **Tanpa Load Balancing Bawaan**

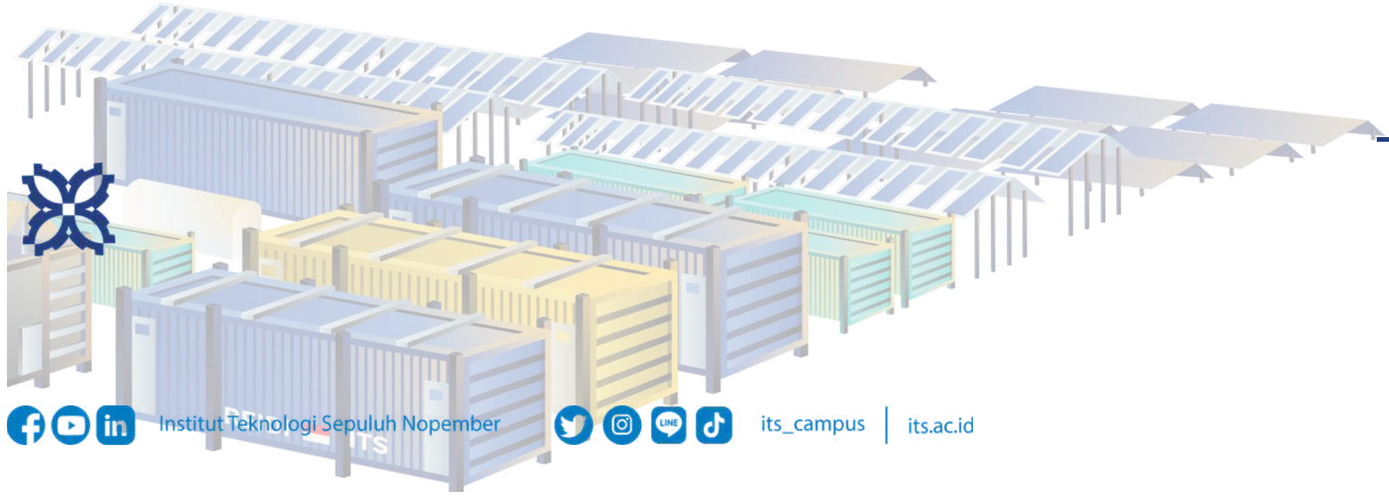
hanya mengejar fitness, sehingga beban VM bisa timpang.

- **Komputasi Berat**

semua kandidat dihitung ulang tiap iterasi, dan hasil bergantung pada ukuran populasi serta jumlah iterasi.

- **Pengujian Terbatas**

paper hanya menguji lewat simulasi dengan task independen dan non-preemptive.



Kelemahan SA

- **Waktu convergence Lambat:**
  - mengharuskan evaluasi berbagai konfigurasi secara berurutan. Dalam lingkungan cloud, itu memakan waktu beberapa menit untuk setiap iterasi, sehingga keseluruhan proses penyesuaian (tuning) menjadi sangat lambat.
- **Biaya Infrastruktur yang Tinggi:**
  - Karena algoritme ini sengaja menguji konfigurasi yang kurang optimal atau "lebih buruk" pada tahap awal untuk mengeksplorasi lanskap yang ada, hal ini dapat menurunkan kinerja aplikasi untuk sementara atau memboroskan sumber daya cloud yang mahal selama fase penyesuaian (tuning).
- **Sensitivitas Hiperparameter:**
  - Keberhasilan sangat bergantung pada penyesuaian parameternya sendiri, yaitu "suhu" awal, laju pendinginan, dan jumlah iterasi untuk setiap penurunan suhu. Pengaturan yang tidak tepat dapat menyebabkan algoritma melewatkan konfigurasi terbaik atau membutuhkan waktu terlalu lama untuk dijalankan.
- **Tidak ada Memori:**
  - SA standar tidak menyimpan memori mengenai konfig yang sukses sebelumnya. Akibatnya, bisa saja secara tidak sengaja menguji kembali status buruk yang serupa, kecuali jika dimodifikasi dengan menambahkan daftar tabu atau riwayat.

Cara Kerja JAYA

1. **Inisiasi Populasi**

JAYA memulai proses dengan membuat beberapa kandidat solusi, yaitu beberapa kemungkinan pemetaan *task* ke VM yang nantinya akan dibandingkan untuk memperoleh pemetaan terbaik. Di mana **nilai awal setiap kandidat** dibangkitkan secara acak menggunakan rumus:

$$X_0^k = X_{min} + (X_{max} - X_{min}) \times r$$

Di mana:

- X_0^k = Nilai awal kandidat solusi,
- X_{min} = Batas minimum nilai yang diperbolehkan,
- X_{max} = Batas maksimum nilai yang diperbolehkan,
- r = Bilangan acak dengan rentang 0 - 1.

Dengan X_{min} dan X_{max} ditentukan dari ruang solusi yang diperbolehkan, yakni batas penempatan *task* sesuai jumlah VM yang tersedia, sehingga tidak ada kandidat solusi yang merepresentasikan penempatan pada VM yang tidak ada.

Cara Kerja JAYA

2. **Evaluasi *Fitness***

Setiap kandidat solusi kemudian diberi **nilai *fitness***, yakni nilai yang menunjukkan seberapa baik pemetaan *task* dan VM tersebut memenuhi tujuan optimasi. *Fitness function* tidak memiliki satu rumus tetap dalam JAYA, karena disesuaikan dengan masalah yang dioptimasi, adapun salah satu rumusnya meliputi:

a. Hitung ***Completion Time* (CT)** dari pasangan *task*-VM, dengan rumusnya:

$$CT_{ij} = \frac{Len_i}{PT_j}$$

Di mana:

- CT_{ij} = Waktu penyelesaian *task* ke-i pada VM ke-j,
- Len_i = Jumlah instruksi pada *task* ke-i (MI),
- PT_j = Kemampuan pemrosesan VM ke-j.

Cara Kerja JAYA

2. **Evaluasi *Fitness***

   b. Hitung ***Execution Time* (ET)** dari setiap VM, dengan menjumlahkan CT seluruh *task* yang dialokasikan ke VM tersebut, di mana rumusnya:

$$ET_j = \sum_{i=1}^{n} X_{ij} \times CT_{ij}$$

Di mana:

- ET_j = Total waktu eksekusi VM ke-j,
- n = Jumlah *task*,
- $X_{ij} = \begin{cases} 1, \text{ jika } task \text{ ke-i dialokasikan ke VM ke-j,} \\ 0, \text{ jika tidak.} \end{cases}$

Cara Kerja JAYA

2. **Evaluasi *Fitness***

c. Hitung komponen fitness F1 dan F2 menggunakan nilai ET yang telah diperoleh dari setiap kandidat pemetaan task-VM untuk mengevaluasi performa kandidat berdasarkan empat metrik, yakni ***Makespan*** **(MS)**, ***Average Resource Utilization*** **(ARU)**, ***Average Throughput*** **(AT)**, dan ***Energy Consumption*** **(EC)**, di mana rumusnya:

$$MS = max\left(ET_j\right)$$

$$F_1 = \frac{1}{MS} \times ARU \times AT$$

$$F_2 = \sum_{j-1}^{m} EC_j$$

Di mana:

- MS = Waktu penyelesaian terlama di antara seluruh VM,
- ARU = Rata-rata tingkat pemanfaatan VM,
- AT = Rata-rata *throughput* sistem,
- EC_j = Energi yang dikonsumsi VM ke-j,
- m = Jumlah VM.

Dengan F_1 menilai performa waktu dan pemanfaatan resource, sedangkan F_2 menghitung total konsumsi energi kandidat tersebut.

Cara Kerja JAYA

2. **Evaluasi *Fitness***

d. Hitung **nilai *fitness* akhir** dengan menggabungkan kedua komponen yang diperoleh menjadi satu nilai *fitness*, rumusnya:

$$F = (F_1 \times \omega_1) + (F_2 \times \omega_2)$$

Di mana:

- F = Nilai *fitness* kandidat pemetaan *task*-VM,
- ω_1 = Bobot komponen F_1, dengan rentang 0 - 1,
- ω_2 = Bobot komponen F_2, dengan rentang 0 - 1.

Seluruh kandidat kemudian dibandingkan berdasarkan nilai F, di mana:

- P_{best} = Kandidat dengan nilai F minimum atau $\underset{k}{argmin} F_k$
- P_{worst} = Kandidat dengan nilai F maksimum atau $\underset{k}{argmax} F_k$

Cara Kerja JAYA

2. **Evaluasi *Fitness***

Semakin kecil nilai *fitness* akhir F , semakin baik kualitas pemetaan antara *task* dengan VM. P_{best} digunakan sebagai acuan untuk mengarahkan kandidat menuju solusi yang lebih baik, sedangkan P_{worst} digunakan sebagai acuan untuk menjauhkan kandidat dari solusi yang buruk. Keduanya kemudian digunakan dalam proses pembaruan kandidat pada iterasi berikutnya.

3. **Pembaruan Kandidat Solusi**

Setelah P_{best} dan P_{worst} diperoleh dari evaluasi *fitness*, setiap kandidat diperbarui agar bergerak mendekati solusi terbaik dan menjauhi solusi terburuk, di mana rumusnya:

$$X_i^{k+1} = X_i^k + r_1 \left(P_{best_i} - |X_i^k|\right) - r_2 \left(P_{worst_i} - |X_i^k|\right)$$

Di mana:

- X_i^k = Nilai penempatan *task* ke-i pada iterasi ke-k,
- X_i^{k+1} = Nilai baru penempatan *task* ke-i setelah diarahkan mendekati P_{best} dan menjauhi P_{worst} ,
- P_{best_i} = Nilai penempatan *task* ke-i pada kandidat solusi terbaik,

Cara Kerja JAYA

3. **Pembaruan Kandidat Solusi**

- P_{worst_i} = Nilai penempatan *task* ke-i pada kandidat solusi terburuk,
- r_1 = Bilangan acak dengan rentang 0 - 1 yang mengatur besar pengaruh P_{best} saat pembaruan,
- r_2 = Bilangan acak dengan rentang 0 - 1 yang mengatur besar pengaruh P_{worst} saat pembaruan,
- k = Indeks iterasi yang sedang berlangsung,
- i = Indeks *task* yang nilainya sedang diperbarui.

Contoh:

Misalkan pada iterasi ke-k, *Task* 2 memiliki:

$X_2^k = 2 \rightarrow$ *Task* 2 saat ini ditempatkan pada VM ke-2,

$P_{best_2} = 3 \rightarrow$ Pada kandidat terbaik, *Task* 2 ditempatkan pada VM ke-3,

$P_{worst_2} = 1 \rightarrow$ Pada kandidat terburuk, *Task* 2 ditempatkan pada VM ke-1,

$r_1, r_2 \rightarrow$ Seimbang, yakni masing-masing bernilai 0,5.

Cara Kerja JAYA

3. **Pembaruan Kandidat Solusi**

Maka:

$$X_2^{k+1} = 2 + 0,5\,(3 - |2|) - 0,5(1 - |2|)$$
$$X_2^{k+1} = 2 + 0.5(1) - 0.5(-1)$$
$$X_2^{k+1} = 3$$

Artinya, setelah proses pembaruan JAYA, nilai penempatan *Task* 2 berubah dari VM ke-2 menjadi VM ke-3, yakni bergerak menuju P_{best} dan menjauhi P_{worst} .

4. **Seleksi Solusi**

Setelah kandidat baru X_i^{k+1} diperoleh dari proses pembaruan, nilai *fitness* kandidat baru dibandingkan dengan kandidat sebelumnya, yakni X_i^k . Adapun syaratnya adalah:

$$F\left(X^{k+1}\right) < F\left(X^k\right)$$

Dengan $F\left(X^{k+1}\right)$ merupakan nilai *fitness* kandidat setelah pembaruan dan $F\left(X^k\right)$ merupakan nilai *fitness* kandidat sebelum.

Cara Kerja JAYA

3. **Pembaruan Kandidat Solusi**

Maka:

$$X_2^{k+1} = 2 + 0,5\,(3 - |2|) - 0,5(1 - |2|)$$
$$X_2^{k+1} = 2 + 0.5(1) - 0.5(-1)$$
$$X_2^{k+1} = 3$$

Artinya, setelah proses pembaruan JAYA, nilai penempatan *Task* 2 berubah dari VM ke-2 menjadi VM ke-3, yakni bergerak menuju P_{best} dan menjauhi P_{worst} .

4. **Seleksi Solusi**

Setelah kandidat baru X_i^{k+1} diperoleh dari proses pembaruan, nilai *fitness* kandidat baru dibandingkan dengan kandidat sebelumnya, yakni X_i^k . Adapun syaratnya adalah:

$$F\left(X^{k+1}\right) < F\left(X^k\right)$$

Dengan $F\left(X^{k+1}\right)$ merupakan nilai *fitness* kandidat setelah pembaruan dan $F\left(X^k\right)$ merupakan nilai *fitness* kandidat sebelum.

Cara Kerja JAYA

4. **Seleksi Solusi**

Di mana, Jika nilai *fitness* kandidat baru lebih baik, yakni ditunjukkan dengan X^{k+1} memiliki nilai yang lebih kecil dibandingkan dengan X^{k}, maka X^{k+1} diterima dan menggantikan kandidat sebelumnya. Jika tidak, maka X^{k} akan tetap dipertahankan. Tujuan tahap ini adalah **memastikan bahwa pembaruan kandidat hanya** dipertahankan apabila **menghasilkan solusi yang lebih baik**, sehingga kualitas populasi tidak menurun pada iterasi berikutnya.

5. **Iterasi**

Setelah tahap seleksi selesai, kandidat yang dipertahankan menjadi populasi untuk iterasi berikutnya. Seluruh kandidat kemudian dievaluasi kembali untuk menentukan P_{best} dan P_{worst} yang baru. Adapun proses yang diulang meliputi:

$$Evaluasi\ \ Fitness \rightarrow Mendapatkan\ \ P_{best}, P_{worst} \rightarrow Pembaruan\ \ Kandidat \rightarrow Seleksi\ Solusi$$

Pada setiap iterasi, nilai P_{best} dan P_{worst} dapat berubah karena kualitas kandidat solusi juga berubah. Proses JAYA dihentikan jika posisi P_{best} dan P_{worst} sudah tidak berubah lagi, yang menunjukkan bahwa proses pembaruan tidak lagi menghasilkan perubahan. Setelah kondisi berhenti tercapai, P_{best} menjadi solusi akhir yang dipilih, yang menunjukkan **pemetaan *task*-VM terbaik yang diperoleh selama proses optimasi**.

Cara Kerja SA

Masalah yang diselesaikan oleh SA dimisalkan seperti memiliki sekumpulan task yang dijalankan beberapa komputer. Tiap komputer punya kecepatan berbeda, sehingga tiap task membutuhkan waktu komputasi berbeda di masing-masing komputer. Tujuan kita adalah membagi tugas-tugas tersebut ke komputer yang sesuai sehingga tidak ada satu komputer yang terlalu lama menyelesaikan seluruh tugas yang dibebankan kepadanya. SA digunakan untuk mencari solusi yang baik dalam waktu yang wajar, tanpa harus memeriksa semua kemungkinan.

1. Menghitung Beban Node

$$ETC(i,j) = \text{waktu eksekusi tugas } t_i \text{ pada node } n_j$$

Contoh:

Task	Node 1	Node 2	Node 3
t1	4	6	9
t2	7	3	5
t3	8	4	2
t4	5	7	4

- ETC(1,1)=4: tugas t1 memerlukan waktu 4 satuan pada node 1.
- ETC(2,2)=3: tugas t2 memerlukan waktu 3 satuan pada node 2.
- ETC(3,3)=2: tugas t3 memerlukan waktu 2 satuan pada node 3.

Cara Kerja SA

2. Representasi Solusi

Representasi ini memudahkan perubahan penempatan tugas dan perhitungan biaya. Fungsinya juga menjadi titik awal pencarian.

$$S = [s_1, s_2, \ldots, s_Y]$$

- Y = jumlah tugas.
- s_i = nomor node tempat tugas t_i dikerjakan.

Contoh:

$$S = [1, 2, 3, 1]$$

- t1 dikerjakan node 1.
- t2 dikerjakan node 2.
- t3 dikerjakan node 3.
- t4 dikerjakan node 1.

Cara Kerja SA

3. Total waktu eksekusi tugas-tugas

$$C_j(S) = \sum_{i=1}^{Y} ETC(i,j)\mathbf{1}(s_i = j)$$

Fungsi indikator $\mathbf{1}(s_i = j)$ bernilai 1 jika tugas i ditempatkan pada node j, dan 0 jika tidak.

Dengan kata lain, kita hanya menjumlahkan waktu tugas yang benar-benar dialokasikan ke node tersebut.

Contoh:

$$C_1 = ETC(1,1) + ETC(4,1)$$

$$C_1 = 4 + 5 = \boxed{9}$$

Cara Kerja SA

3. Menghitung Cost Solution Funtion

$$\boxed{f(S) = \max_{j=1,\ldots,N} C_j(S)}$$

Keterangan:

$f(S)$ = cost atau makespan solusi S.

$C_j(S)$ = total waktu tugas yang dialokasikan ke node j.

N = jumlah node

Contoh: $f(S_0) = \max(9, 3, 2)$

$$\boxed{f(S_0) = 9}$$

Mengapa menggunakan nilai maksimum? Karena seluruh tugas baru selesai ketika node dengan beban terbesar telah menyelesaikan tugasnya, dengan asumsi tugas independen dan setiap node menjalankan tugasnya secara berurutan.

Cara Kerja SA

4. Menentukan temperatur awal

Temperatur awal T0 mengendalikan seberapa berani SA menerima solusi yang lebih buruk.

- Temperatur tinggi: solusi yang lebih buruk relatif mudah diterima.
- Temperatur rendah: solusi yang lebih buruk semakin sulit diterima.

$$\boxed{T_0 = -\frac{\overline{\Delta C}}{\ln(HEC)}}$$

$\overline{\Delta C}$ = rata-rata kenaikan cost dari perpindahan yang memperburuk solusi.

HEC = target probabilitas penerimaan solusi yang lebih buruk.

$\ln$ = logaritma natural

Contoh

dari sampel perpindahan yang memperburuk solusi didapatkan rata-rata kenaikan cost sebesar 2 satuan. parameter target yang dipilih adalah H=0.9.

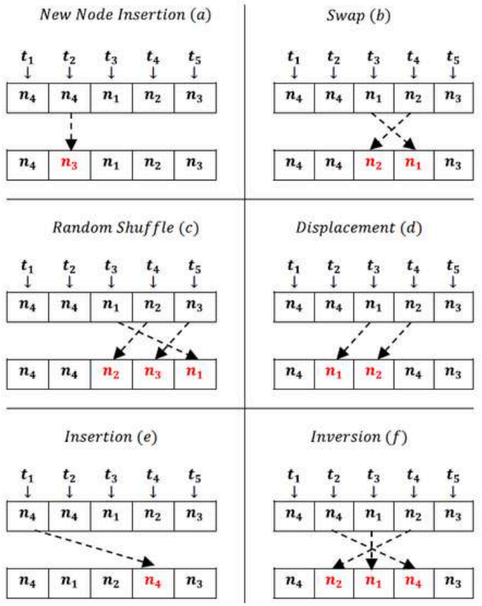
$$T_0 = -\frac{2}{\ln(0{,}9)}$$

$$\boxed{T_0 \approx 18{,}98}$$

Cara Kerja SA

5. Membentuk Neighbor Solution

SA tidak mengganti semua tugas secara sembarangan pada setiap iterasi. Ia menggunakan operasi yang disebut perturbation, yaitu perubahan terhadap solusi yang sudah ada.



Cara Kerja SA

6. Menghitung Perubahan Cost ΔC

Setelah neighbor terbentuk, SA menghitung selisih antara cost solusi baru dan cost solusi saat ini

$$\boxed{\Delta C = C_{\text{next}} - C_{\text{cur}}}$$

Interpretasi

Kondisi	Makna	Keputusan
$\Delta C < 0$	Cost turun	Terima solusi baru
$\Delta C = 0$	Cost tetap	Terima solusi baru
$\Delta C > 0$	Cost naik	Uji probabilitas penerimaan

Cara Kerja SA

7. Menghitung probabilitas penerimaan solusi yang lebih buruk

Inilah bagian yang membedakan Simulated Annealing dari metode pencarian yang selalu memilih solusi terbaik pada setiap langkah.

$$\boxed{P = \exp\left(-\frac{\Delta C}{T}\right)} \quad \text{untuk } \Delta C > 0$$

- P = probabilitas penerimaan.
- ΔC = kenaikan cost.
- T = temperatur saat ini.
- $\exp(x) = e^x$, dengan $e \approx 2{,}71828$.

Kemudian algoritma menghasilkan bilangan acak r antara 0 dan 1.

$$\boxed{\begin{cases} \text{Terima}, & r < P \\ \text{Tolak}, & r \geq P \end{cases}}$$

Menerima solusi yang lebih buruk apabila bilangan acak kurang dari $\exp(-\Delta C/T)$

Cara Kerja SA

8. Memperbarui solusi saat ini dan solusi terbaik

Jika neighbor diterima, solusi saat ini diperbarui menjadi solusi baru:

$$S_{\text{cur}} \leftarrow S_{\text{next}}$$

Kemudian, jika solusi tersebut lebih baik daripada solusi terbaik:

$$f(S_{\text{cur}}) < f(S_{\text{best}})$$

Maka

$$S_{\text{best}} \leftarrow S_{\text{cur}}$$

solusi yang diterima belum tentu menjadi solusi terbaik. SA dapat menerima solusi dengan cost lebih tinggi untuk memperluas pencarian, tetapi solusi terbaik yang pernah ditemukan tetap disimpan. Sebaliknya, jika neighbor ditolak, solusi saat ini tetap seperti semula.

Cara Kerja SA

9. Menurunkan temperatur (cooling schedule)

Setelah sejumlah pencarian pada satu temperatur selesai, SA menurunkan temperaturnya.

$$\boxed{T_{\text{baru}} = \alpha T_{\text{lama}}}$$

- α = cooling rate, dengan nilai antara 0 dan 1.
- T_{lama} = temperatur sebelum penurunan.
- T_{baru} = temperatur setelah penurunan.

Contoh

$\alpha = 0{,}998$

$T_{\text{lama}} = 10$

$$T_1 = 0{,}998(10) = 9{,}98$$

$$T_2 = 0{,}998(9{,}98) = 9{,}96004$$

$$T_3 = 10(0{,}998)^3 \approx 9{,}94012$$

$$P = e^{-4/10} \approx 0{,}6703$$

$$P = e^{-4/1} \approx 0{,}0183$$

Jadi, semakin rendah temperatur, semakin kecil peluang SA menerima solusi yang memperburuk cost.

Cara Kerja SA

10. Mengulangi pencarian sampai kondisi berhenti terpenuhi

SA tidak hanya melakukan satu perubahan. Proses pembentukan neighbor, penghitungan cost, keputusan penerimaan, dan pembaruan solusi diulang berkali-kali.

struktur perulangan:

- Inner loop: mengatur berapa kali solusi tetangga dicoba pada temperatur tertentu.
- Outer loop: mengatur proses penurunan temperatur dan kapan algoritma dihentikan.

Kondisi penghentian dasarnya adalah:

$$T < T_{\text{final}}$$

Dengan $T_{\text{final}} = 0{,}01$, pencarian berakhir ketika temperatur telah turun melewati ambang tersebut, kecuali proses dihentikan lebih awal oleh kondisi lain.

Ketika algoritma selesai, keluaran yang digunakan adalah S_{best}

Referensi

Kaushik Mishra, Jharashree Pati, Santosh Kumar Majhi (2020). *A dynamic load scheduling in IaaS cloud using binary JAYA algorithm*. Journal of King Saud University - Computer and Information Sciences, 34 (2022), 4914 - 4930.

Esra Celik, Deniz Dal (2021), *A novel simulated annealing-based optimization approach for clusterbased task scheduling*, Cluster Computing (2021) 24, 2927 - 2956.

TERIMA KASIH

Kelompok 4 Kelas C